

Vitest Custom Hook Testing Guide

A hands-on, example-driven walkthrough of custom React hook test scenarios

Learn how to test React custom hooks with Vitest and React Testing Library. This guide follows the same running example — the **useUsers()** hook from the User Management feature — covering public API, state transitions, edge cases, race conditions, cleanup and production risks.

Audience	QA freshers & frontend developers
Running example	useUsers() hook — User Management feature
Tools	Vitest + React Testing Library + TanStack Query
Version	1.0

Contents

1. How to Use This Guide	3
2. The Running Example — useUsers() Hook	4
3. Hook Information	6
4. Test Scenarios	8
5. Edge & Failure Cases	12
6. Production Risks	16
7. Review Checklist	18
8. Quick Reference Cards	19

1. How to Use This Guide

This guide teaches freshers how to test **custom React hooks** using **Vitest** and **React Testing Library**. It is the companion to the Manual Testing Guide — same running example, but now we test the *hook layer* (the data-fetching and state-management logic) rather than the UI.

Who is this for?

- QA freshers transitioning to frontend test automation
- Frontend developers writing their first hook tests
- Manual testers learning Vitest to level up

What is a custom hook?

A custom hook is a JavaScript function that starts with `use` and encapsulates reusable stateful logic. In our architecture, hooks sit in **Layer 3** — they call the API layer, pass results through the mapper, and return domain types to the UI. They are the bridge between "fetching data" and "rendering data".

```
// The hook we will test throughout this guide
export function useUsers() {
  return useQuery({
    queryKey: ['users'],
    queryFn: async () => {
      const raw = await fetchUsers(); // Layer 1: API
      return mapUsersFromApi(raw);    // Layer 2: Mapper
    },
  });
}
```

Why test hooks separately?

Reason	Benefit
Hooks contain business logic (fetching, mapping, caching)	Logic bugs caught before they reach the UI
Hooks are reusable across multiple components	A bug in the hook affects every consumer
Hooks manage async state (loading, error, success)	State transitions can be tested deterministically
Hooks interact with external libraries (TanStack Query)	Integration points verified in isolation

TIP

If you have not read the *Manual Testing Guide for Freshers*, start there. This guide assumes you understand the User Management feature and the concept of test scenarios. Here we apply the same thinking to hook-level automated tests.

2. The Running Example — useUsers() Hook

We continue with the **User Management** feature from the manual testing guide. The feature-sliced architecture has four layers: `api/`, `model/`, `hooks/`, `ui/`. We are testing the hook layer — specifically the `useUsers()` and `useUser()` hooks.

2.1 The hooks under test

<code>useUsers()</code>	Returns all users — loading state, error state, data (<code>User[]</code>), refetch function
<code>useUser(id)</code>	Returns a single user by ID — loading, error, data (<code>User undefined</code>)
<code>useCreateUser()</code>	Returns a mutation — mutate function, <code>isPending</code> , <code>isError</code> , error
<code>useDeleteUser()</code>	Returns a mutation — mutate function, <code>isPending</code> , <code>isError</code> , error

2.2 Full source code of useUsers()

```
// features/users/hooks/useUsers.ts
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { fetchUsers, fetchUserById, createUser, deleteUser } from '../api/usersApi';
import { mapUsersFromApi, mapUserFromApi } from '../model/mappers';
import type { User } from '../model/types';

export function useUsers() {
  return useQuery({
    queryKey: ['users'],
    queryFn: async (): Promise<User[]> => {
      const raw = await fetchUsers();
      return mapUsersFromApi(raw);
    },
  });
}

export function useUser(userId: number) {
  return useQuery({
    queryKey: ['users', userId],
    queryFn: async (): Promise<User> => {
      const raw = await fetchUserById(userId);
      return mapUserFromApi(raw);
    },
    enabled: !!userId,
  });
}

export function useCreateUser() {
  const queryClient = useQueryClient();
  return useMutation({
```

```

    mutationFn: async (input: { name: string; email: string }) => {
      const raw = await createUser(input);
      return mapUserFromApi(raw);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['users'] });
    },
  });
}

export function useDeleteUser() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async (userId: number) => {
      await deleteUser(userId);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['users'] });
    },
  });
}

```

2.3 The test setup

```

// features/users/hooks/useUsers.test.ts
import { renderHook, waitFor } from '@testing-library/react';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { useUsers, useUser, createUser, deleteUser } from '../useUsers';
import * as usersApi from '../api/usersApi';

function createWrapper() {
  const queryClient = new QueryClient({
    defaultOptions: { queries: { retry: false } },
  });
  return ({ children }: { children: React.ReactNode }) => (
    <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
  );
}

```

WHY THIS EXAMPLE?

The `useUsers()` hook touches every concern a custom hook can have: async data fetching, mapping/transformation, loading/error states, caching, mutations with cache invalidation, and conditional enabling. If you can test this hook thoroughly, you can test almost any data-fetching hook.

3. Hook Information

Before writing any test, you must understand the hook's **contract**: what it accepts, what it returns, and what guarantees it makes. This section captures the hook's public API so that test scenarios map directly to observable behaviours.

3.1 Hook Summary

Purpose	Fetch, cache, and provide user data to UI components. Encapsulates TanStack Query logic so UI components never call the API directly.
Layer	Layer 3 — Hooks (features/users/hooks/)
Dependencies	TanStack Query (useQuery, useMutation, useQueryClient), API layer (fetchUsers, fetchUserById, createUser, deleteUser), Model layer (mapUsersFromApi, mapUserFromApi)
Consumers	UserList.tsx, UserDetails.tsx, AddUserModal.tsx — all import from the barrel (features/users/index.ts)

3.2 Public API

The barrel export `features/users/index.ts` exposes these hooks to the rest of the app:

Hook	Input	Return Shape	Key Behaviours
<code>useUsers()</code>	none	<code>{ data, isLoading, isError, error, refetch }</code>	Fetches all users on mount; caches under key <code>['users']</code> ; returns <code>User[]</code> (domain types, never raw API types)
<code>useUser(id)</code>	<code>id: number</code>	<code>{ data, isLoading, isError, error }</code>	Fetches single user; disabled when <code>id</code> is falsy (enabled: <code>!!id</code>); caches under <code>['users', id]</code>
<code>useCreateUser()</code>	none	<code>{ mutate, isPending, isError, error, data }</code>	Calls <code>createUser</code> API; on success invalidates <code>['users']</code> cache to trigger refetch
<code>useDeleteUser()</code>	none	<code>{ mutate, isPending, isError, error }</code>	Calls <code>deleteUser</code> API; on success invalidates <code>['users']</code> cache

3.3 State machine

Each query hook has three primary states. Understanding these is essential for writing test scenarios:

State	When	Properties	UI impact
-------	------	------------	-----------

Loading	First fetch, no cached data	isLoading = true, data = undefined	Skeleton / spinner shown
Success	API returned data, mapper succeeded	isLoading = false, data = User[], isError = false	User list rendered
Error	API threw or mapper threw	isLoading = false, isError = true, error = Error	Error message shown

KEY IDEA

Every test scenario you write should target one of these states or a transition between them. If a scenario does not test a state or a transition, it is probably testing the wrong thing.

EXAMPLE — PUBLIC API MAPPING

When `UserList.tsx` calls `useUsers()`, it destructures `{ data, isLoading, isError }`. The component renders a skeleton when `isLoading`, an error banner when `isError`, and the user table when data is present. Each of these three branches is a test scenario.

4. Test Scenarios

Each scenario tests **one public behaviour** of the hook. The golden rule: **one test = one behaviour = one assertion group**. Do not test loading, success, and error in the same test — split them.

4.1 How to write a good hook test

1. **Mock the API layer** — never make real network calls. Use `vi.mock()` or `vi.spyOn()`.
2. **Render the hook** — use `renderHook` from React Testing Library with a `QueryClientProvider` wrapper.
3. **Wait for async** — use `waitFor` to wait for state transitions (loading → success/error).
4. **Assert the return shape** — check `data`, `isLoading`, `isError`, `error` match expectations.
5. **Verify mocks were called** — confirm the API function was called with the right arguments.

COMMON MISTAKE

Forgetting to wrap the hook in a `QueryClientProvider`. The hook uses `useQuery` which requires a `QueryClient` context. Without the wrapper, the test crashes with "No QueryClient set". Always use the `createWrapper()` helper.

4.2 Test scenarios — `useUsers()`

#	Scenario	Setup / Steps	Expected Result	Vitest Code Snippet
T-01	Returns loading state initially	Render hook with mocked <code>fetchUsers</code> (not yet resolved)	<code>isLoading = true</code> , <code>data = undefined</code> , <code>isError = false</code>	<code>expect(result.current.isLoading).toBe(true)</code>
T-02	Returns mapped user data on success	Mock <code>fetchUsers</code> to resolve with raw API data; <code>waitFor</code>	<code>isLoading = false</code> , <code>data = User[]</code> (domain types), <code>isError = false</code> ; mapper was called	<code>await waitFor(() => expect(result.current.data).toHaveLength(2))</code>
T-03	Returns error state on API failure	Mock <code>fetchUsers</code> to reject with <code>Error("500")</code> ; <code>waitFor</code>	<code>isLoading = false</code> , <code>isError = true</code> , <code>error.message = "500"</code> , <code>data = undefined</code>	<code>await waitFor(() => expect(result.current.isError).toBe(true))</code>
T-04	Data is mapped through mapper,	Mock <code>fetchUsers</code> to return raw shape (nested address); check data shape	<code>data[0].displayName</code> (not <code>data[0].name</code>); <code>data[0].location</code> is a	<code>expect(result.current.data[0].displayName).to</code>

	not raw API types		flat string (not nested object)	
T-05	refetch re-calls fetchUsers	Render hook; call result.current.refetch(); waitFor	fetchUsers called 2 times total; data refreshed	await result.current.refetch(); expect(fetchUsers).toHaveBeenCalledTimes(2)
T-06	Cache prevents duplicate fetch on re-render	Render hook; re-render same hook; check call count	fetchUsers called only once (served from cache on second render)	rerender(); expect(fetchUsers).toHaveBeenCalledTimes(1)

4.3 Worked example — T-02 (success state)

```
// T-02: Returns mapped user data on success
it('returns mapped user data on success', async () => {
  // Arrange — mock the API layer
  const mockRawUsers = [
    { id: 1, name: 'John', address: { city: 'NYC', street: '5th Ave' },
      company: { name: 'Acme' } },
  ];
  vi.spyOn(usersApi, 'fetchUsers').mockResolvedValue(mockRawUsers);

  // Act — render the hook
  const { result } = renderHook(() => useUsers(), {
    wrapper: createWrapper(),
  });

  // Assert — wait for success state
  await waitFor(() => {
    expect(result.current.isLoading).toBe(false);
  });

  expect(result.current.isError).toBe(false);
  expect(result.current.data).toHaveLength(1);
  // Verify mapping: displayName, not name; location is flat string
  expect(result.current.data[0].displayName).toBe('John');
  expect(result.current.data[0].location).toBe('NYC, 5th Ave');
  expect(result.current.data[0].company).toBe('Acme');
});
```

EXAMPLE WALKTHROUGH — T-02

What we are testing: When fetchUsers resolves, the hook should pass the raw data through the mapper and return domain-typed User objects — not raw API shapes.

Key assertions: (1) data has 1 item, (2) data[0].displayName = "John" (not data[0].name), (3) data[0].location = "NYC, 5th Ave" (flattened, not nested), (4) data[0].company = "Acme" (simplified, not the whole company object).

Why this matters: If the mapper is bypassed or broken, UI components receive raw API types — they break when the backend schema changes. This test enforces the anti-corruption layer.

4.4 Test scenarios — useUser(id)

#	Scenario	Setup / Steps	Expected Result
T-07	Fetches user by ID	Render useUser(1); mock fetchUserById to resolve	data = User with id=1; isLoading transitions true → false
T-08	Does not fetch when id is falsy	Render useUser(0); check fetchUserById not called	fetchUserById not called; isLoading = false (enabled: !!id prevents query)
T-09	Does not fetch when id is undefined	Render useUser(undefined as any)	fetchUserById not called; query is disabled
T-10	Returns error on 404 (user not found)	Mock fetchUserById to reject with 404 error	isError = true; error message contains "not found"

4.5 Test scenarios — useCreateUser() & useDeleteUser()

#	Scenario	Setup / Steps	Expected Result
T-11	Create user — success	Call mutate({ name, email }); mock createUser to resolve	isPending transitions true → false; data = new User; cache invalidated
T-12	Create user — API error	Mock createUser to reject with "Email already exists"	isError = true; error.message = "Email already exists"; data = undefined
T-13	Create user — invalidates users cache	Call mutate; check queryClient.invalidateQueries called	invalidateQueries called with { queryKey: ['users'] }
T-14	Delete user — success	Call mutate(1); mock deleteUser to resolve	isPending transitions true → false; cache invalidated
T-15	Delete user — API error	Mock deleteUser to reject with 403	isError = true; error contains 403; users cache not invalidated

```
// T-13: Create user invalidates users cache
it('invalidates users cache after successful create', async () => {
  vi.spyOn(usersApi, 'createUser').mockResolvedValue({ id: 99, name: 'Jane' });
  const invalidateSpy = vi.spyOn(QueryClient.prototype, 'invalidateQueries');

  const { result } = renderHook(() => useCreateUser(), {
    wrapper: createWrapper(),
  });
});
```

```
result.current.mutate({ name: 'Jane', email: 'jane@ex.com' });

await waitFor(() => {
  expect(invalidateSpy).toHaveBeenCalledWith({ queryKey: ['users'] });
});
});
```

TIP

Always test cache invalidation after mutations. This is the most commonly forgotten test. If invalidation does not fire, the UI shows stale data after create/delete — the same bug as "user created but not visible in the list" from the manual guide.

5. Edge & Failure Cases

Edge cases for hooks are different from manual UI edge cases. Here we test the hook's behaviour when inputs are weird, data is unexpected, async timing is tricky, or React lifecycle events cause problems. This is where senior test engineers add the most value.

MINDSET SHIFT

In manual testing, you test what the *user* does. In hook testing, you test what the *consumer component* and the *async environment* do. The "user" of a hook is a React component — and components can render, unmount, re-render, and pass bad props in ways users never would.

5.1 Edge case scenarios — full table

#	Category	How to test	Expected behaviour	Example (useUsers)
E-01	null / undefined inputs	Render useUser(null) or useUser(undefined)	Query disabled; no API call; no crash	useUser(null) → fetchUserById not called; data = undefined; isLoading = false
E-02	Empty values	Mock fetchUsers to resolve with [] (empty array)	data = [], isLoading = false, isError = false	fetchUsers returns [] → data = [], not undefined; UI can show empty state
E-03	Invalid values	Render useUser(-1) or useUser(NaN)	Query disabled or API called with -1; error handled gracefully	useUser(-1) → either disabled (if !!-1 = true, fetch happens); API returns 404 → isError = true
E-04	Large datasets	Mock fetchUsers to resolve with 10,000 items	data has 10,000 items; no timeout; no memory error; mapper processes all	fetchUsers returns 10k raw users → data has 10k mapped Users; test completes in <1s
E-05	Duplicate data	Mock fetchUsers to return two users with same id	Hook returns both; no deduplication (unless designed); UI handles or doesn't	Two raw users with id=1 → data has 2 items; test documents current behaviour
E-06	Missing fields	Mock fetchUsers to return user with missing company field	Mapper handles gracefully (default value) or throws (documented)	raw user without company → mapper returns company = " or throws TypeError; test asserts which
E-07	Loading state	Check isLoading before promise resolves	isLoading = true synchronously on first render	renderHook → immediately check

				<code>result.current.isLoading === true</code>
E-08	Error state	Mock <code>fetchUsers</code> to reject; check error shape	<code>isError = true</code> ; error is Error instance with message; data = undefined	<code>fetchUsers</code> rejects with <code>new Error('Network')</code> → <code>error.message = 'Network'</code>
E-09	Empty response	Mock <code>fetchUsers</code> to resolve with null (not <code>[]</code>)	Hook handles null → either data = <code>[]</code> or throws; test documents behaviour	<code>fetchUsers</code> returns null → mapper throws or returns <code>[]</code> ; test asserts which
E-10	Retry	Configure <code>QueryClient</code> with <code>retry: 2</code> ; mock <code>fetchUsers</code> to fail once then succeed	<code>fetchUsers</code> called 2 times; final state = success	1st call rejects, 2nd call resolves → data = <code>User[]</code> ; <code>fetchUsers</code> called 2x
E-11	Multiple concurrent requests	Render <code>useUsers()</code> and <code>useUser(1)</code> simultaneously	Both hooks work independently; no shared state contamination	Two hooks in same wrapper → both fetch; both return correct data
E-12	Out-of-order responses	Mock <code>useUser(1)</code> slow, <code>useUser(2)</code> fast; render both; switch id from 1 to 2	Last rendered result wins; no stale data from old request	Render <code>useUser(1)</code> then switch to <code>useUser(2)</code> → data = user 2, not user 1
E-13	Rapid state updates	Call <code>mutate()</code> 3 times rapidly	All mutations processed; no race condition; final state consistent	3x <code>mutate create</code> → 3 API calls; cache invalidated 3x; no crash
E-14	Cleanup on unmount	Render hook; unmount before promise resolves	No state update after unmount; no "Can't perform React state update on unmounted component" warning	<code>renderHook</code> → <code>unmount()</code> immediately → no console warnings; <code>fetch</code> may still resolve but no state update
E-15	React Strict Mode	Wrap test in <code>React.StrictMode</code> (double-invokes effects)	<code>fetchUsers</code> called 2x (Strict Mode); data correct; no duplicate final state	StrictMode wrapper → <code>fetchUsers</code> called 2x; data still correct; no error
E-16	Consumer misuse	Call <code>useUsers()</code> without wrapper (no <code>QueryClient</code>)	Throws clear error: "No <code>QueryClient</code> set"	<code>renderHook</code> without wrapper → throws Error containing "QueryClient"
E-17	Race conditions	Render <code>useUser(1)</code> ; immediately switch to <code>useUser(2)</code> ; mock 1 slow, 2 fast	data = user 2 (last request wins); <code>TanStack Query</code> handles via query key	Switch id 1 → 2 before 1 resolves → data = user 2; user 1 result discarded

EXAMPLE — E-14: CLEANUP ON UNMOUNT

Why this matters: If a hook updates state after the component unmounts, React logs a warning and may cause memory leaks in production. TanStack Query handles this internally, but custom hooks with `useEffect` + `useState` can still have this bug.

```
// E-14: Cleanup on unmount – no warning after unmount
it('does not update state after unmount', async () => {
  const slowPromise = new Promise(() => {}); // never resolves
  vi.spyOn(usersApi, 'fetchUsers').mockReturnValue(slowPromise);

  const spy = vi.spyOn(console, 'error').mockImplementation(() => {});

  const { unmount } = renderHook(() => useUsers(), {
    wrapper: createWrapper(),
  });

  unmount(); // unmount while still loading

  // Wait a tick to see if any state update fires
  await new Promise(r => setTimeout(r, 100));

  expect(spy).not.toHaveBeenCalledWith(
    expect.stringContaining('unmounted component')
  );
  spy.mockRestore();
});
```

5.2 Race condition deep dive — E-17

Race conditions are the hardest bugs to find in async hooks. They occur when multiple async operations are in flight and the response order does not match the request order.

```
// E-17: Race condition – last request wins
it('returns data for the latest id when id changes rapidly', async () => {
  vi.spyOn(usersApi, 'fetchUserById').mockImplementation((id) => {
    // User 1 is slow, user 2 is fast
    if (id === 1) return new Promise(r => setTimeout(() => r({ id: 1, name: 'Slow' })),
    100));
    return Promise.resolve({ id: 2, name: 'Fast' });
  });

  const { result, rerender } = renderHook(
    ({ id }) => useUser(id),
    { wrapper: createWrapper(), initialProps: { id: 1 } }
  );

  rerender({ id: 2 }); // switch to user 2 before user 1 resolves

  await waitFor(() => {
    expect(result.current.data?.displayName).toBe('Fast');
  });

  // Even after user 1's slow response arrives, data should still be user 2
  await new Promise(r => setTimeout(r, 200));
});
```

```
expect(result.current.data?.displayName).toBe('Fast'); // not 'Slow'
});
```

EXAMPLE — E-17 RACE CONDITION

Scenario: UserList component shows user 1's detail. Admin quickly clicks user 2. The hook switches from `useUser(1)` to `useUser(2)`. User 1's API response is slow and arrives after user 2's fast response.

Expected: The hook shows user 2's data. User 1's late response is discarded. TanStack Query handles this via query key switching.

Defect example: A custom hook without TanStack Query uses a simple `useEffect + fetch + setState` pattern. User 1's late response overwrites user 2's data → UI shows wrong user. This is a classic race condition bug.

5.3 React Strict Mode — E-15

React Strict Mode double-invokes effects in development to surface side-effect bugs. TanStack Query is designed to handle this, but custom hooks with manual effects may break.

```
// E-15: React Strict Mode – double effect invocation
it('works correctly under React Strict Mode', async () => {
  const spy = vi.spyOn(usersApi, 'fetchUsers').mockResolvedValue([]);

  const strictWrapper = ({ children }: { children: React.ReactNode }) => (
    <React.StrictMode>
      <QueryClientProvider client={new QueryClient({ defaultOptions: { queries: { retry:
false } } })}>
        {children}
      </QueryClientProvider>
    </React.StrictMode>
  );

  const { result } = renderHook(() => useUsers(), { wrapper: strictWrapper });

  await waitFor(() => {
    expect(result.current.data).toEqual([]);
  });

  // Strict Mode double-invokes; fetchUsers may be called 2x
  // but data should be correct and no error should occur
  expect(result.current.isError).toBe(false);
});
```

TIP

Not every edge case applies to every hook. Review the 17 categories and pick the ones that are relevant. For a simple `useUsers()` query hook, E-01 through E-12 and E-14 are essential. E-13, E-15, E-17 are critical for hooks with dynamic parameters or mutations.

Notes: For the useUsers hook specifically, also consider: (1) what happens when the mapper throws on a single item in a large array — does the whole query fail or just that item? (2) staleTime / gcTime behaviour — does the hook refetch on window focus? (3) What if the API returns 200 but with malformed JSON?

6. Production Risks

Even with all tests passing, certain patterns in hooks create production risks that may not surface in unit tests but can cause real incidents. This section helps you identify and document these risks before release.

6.1 Risk register — useUsers() hook

#	Risk	Severity	Description	Mitigation / Test to add
R-01	Stale data after mutation	High	useCreateUser or useDeleteUser does not invalidate cache → UI shows old list after add/delete	Test T-13 verifies invalidateQueries is called; add integration test with useUsers + useCreateUser together
R-02	Memory leak from uncleaned subscriptions	Medium	If hook uses useEffect with event listeners or WebSocket subscriptions and does not clean up, memory grows	Test E-14 (cleanup on unmount); check for console warnings; profile memory in DevTools
R-03	Race condition on rapid ID changes	High	useUser(id) with rapid id changes → old response overwrites new data	Test E-17 (race condition); use AbortController or rely on TanStack Query's built-in handling
R-04	Unbounded retry causing API flood	Medium	QueryClient retry: 3 + refetchOnWindowFocus: true → on network issues, 3 retries x every focus = many API calls	Configure retry: 1 in production; test retry behaviour (E-10); monitor API rate limits
R-05	Mapper crash on unexpected API shape	High	Backend changes schema (e.g. removes company field) → mapper throws → entire users list fails to load	Test E-06 (missing fields); add defensive defaults in mapper; contract tests against OpenAPI spec
R-06	No error boundary for hook consumers	Medium	If hook throws (not isError, actual throw), consumer component crashes → blank page	Test E-16 (consumer misuse); add Error Boundary around UserList; ensure hook never throws synchronously
R-07	Cache key collision	Low	If two features use queryKey ['users'] with different data shapes, cache serves wrong data	Use namespaced keys: ['users', 'list'] vs ['users', 'detail', id]; test with multiple hooks (E-11)
R-08	Large dataset causing browser freeze	Medium	10,000 users returned → mapper processes all synchronously → main thread blocked for 2s+	Test E-04 (large dataset); consider server-side pagination; virtualize the list in UI

EXAMPLE — R-01: STALE DATA AFTER MUTATION

Risk: Admin creates a new user via `useCreateUser`. The mutation succeeds, but if `invalidateQueries` is missing or broken, the users list still shows the old data. The admin thinks the create failed and clicks `Save again` → duplicate user.

This is the same bug as BUG-UM-001 from the manual testing guide (duplicate user on rapid double-save), but at the hook level. The root cause is missing cache invalidation, not a UI debounce issue.

Mitigation: Test T-13 verifies `invalidateQueries` is called. Additionally, add an integration test that renders both `useUsers()` and `useCreateUser()`, calls `mutate`, and verifies the list refetches.

COMMON MISTAKE

Treating production risks as "someone else's problem". As a tester, it is your job to identify and document risks — even if the fix is a developer's responsibility. A risk that is documented can be mitigated. A risk that is unknown becomes an incident.

6.2 Risk assessment summary

Total risks identified	8
High severity	3 (R-01, R-03, R-05) — must be mitigated before release
Medium severity	4 (R-02, R-04, R-06, R-08) — mitigate in next sprint
Low severity	1 (R-07) — monitor, fix when time permits
Risks with test coverage	All 8 have corresponding test scenarios (T-13, E-14, E-17, E-10, E-06, E-16, E-11, E-04)

7. Review Checklist

Before signing off a hook test suite as complete, go through this checklist. Every item must be ticked. If any item cannot be ticked, there is a gap in test coverage.

7.1 Coverage checklist

- ☐ Every public behaviour has a scenario (useUsers: T-01 to T-06; useUser: T-07 to T-10; useCreateUser: T-11 to T-13; useDeleteUser: T-14 to T-15)
- ☐ Every state transition has a scenario (loading → success, loading → error, idle → pending → success, idle → pending → error)
- ☐ Error paths are covered (API rejection, mapper throw, 404, 403, 500, network error)
- ☐ Edge cases have been considered (all 17 categories reviewed; relevant ones tested)
- ☐ Cleanup behaviour is covered (unmount during loading, Strict Mode double-invoke)
- ☐ No duplicate scenarios (each test verifies one unique behaviour)
- ☐ Another engineer can implement the complete Vitest suite from this document

7.2 Additional quality checks

Check	Question	Status
Test isolation	Does each test clean up mocks and QueryClient? (beforeEach with vi.restoreAllMocks)	Verify
No real network calls	Are all API functions mocked? No accidental fetch to real backend?	Verify
Deterministic tests	Do tests pass consistently? No flaky tests due to timing?	Verify
Descriptive test names	Can you understand what each test does from its name alone?	Verify
Arrange-Act-Assert	Is each test structured with clear Arrange, Act, Assert sections?	Verify
Production risks documented	Are all identified risks from Section 6 documented and assigned?	Verify

7.3 Sign-off summary

Hook under test	useUsers(), useUser(id), useCreateUser(), useDeleteUser()
-----------------	---

Feature	User Management (features/users/hooks/)
Tester	_____
Date	_____
Total scenarios	15 functional (T-01 to T-15) + 17 edge cases (E-01 to E-17) = 32 total
Scenarios executed	_____ passed, _____ failed, _____ skipped
Production risks	_____ identified, _____ mitigated, _____ open
Result	PASS FAIL PASS WITH NOTES
Signature	_____

8. Quick Reference Cards

Cheat sheets you can print and keep at your desk while writing hook tests.

8.1 The 5-step hook testing routine

Step	Action	Question to ask
1	Understand the API	What does the hook accept? What does it return? What states does it have?
2	Test happy path	Does the hook return correct data when the API succeeds?
3	Test states	Is loading true initially? Is error set on failure? Is data undefined before resolve?
4	Test edge cases	What happens with null, empty, large, missing, duplicate, rapid, unmount, Strict Mode?
5	Test side effects	Does mutation invalidate cache? Does unmount clean up? Are mocks called correctly?

8.2 Vitest + RTL cheat sheet

Pattern	Code
Mock API function	<code>vi.spyOn(usersApi, 'fetchUsers').mockResolvedValue([...])</code>
Mock API rejection	<code>vi.spyOn(usersApi, 'fetchUsers').mockRejectedValue(new Error('500'))</code>
Render hook with context	<code>renderHook(() => useUsers(), { wrapper: createWrapper() })</code>
Wait for state change	<code>await waitFor(() => expect(result.current.data).toHaveLength(2))</code>
Check loading initially	<code>expect(result.current.isLoading).toBe(true) (before any await)</code>
Re-render with new props	<code>rerender({ id: 2 })</code>
Unmount hook	<code>unmount()</code> — then assert no console warnings
Verify mock called	<code>expect(fetchUsers).toHaveBeenCalledTimes(1)</code>
Verify mock called with args	<code>expect(fetchUserById).toHaveBeenCalledWith(1)</code>
Cleanup between tests	<code>afterEach(() => { vi.restoreAllMocks(); queryClient.clear() })</code>

8.3 Edge case quick scan

Category	One-line test idea	useUsers example
null / undefined	Pass null/undefined as hook argument	useUser(null) → disabled
Empty	API returns [] or ""	fetchUsers → [] → data = []
Invalid	Pass -1, NaN, wrong type	useUser(-1) → 404 error
Large	API returns 10,000 items	10k users → no freeze
Missing fields	API returns partial object	No company field → mapper handles
Race condition	Switch args before first resolves	id 1 → 2 fast → data = user 2
Cleanup	Unmount before resolve	No console warning
Strict Mode	Wrap in React.StrictMode	Double invoke → still correct

8.4 State transition cheat sheet

From	To	Trigger	Assert
idle	loading	Hook renders (mount)	isLoading = true, data = undefined
loading	success	API resolves	isLoading = false, data = User[], isError = false
loading	error	API rejects	isLoading = false, isError = true, error = Error
success	loading	refetch() called	isLoading = true (or isFetching = true)
error	loading	refetch() called	isLoading = true, previous error cleared
idle	pending	mutate() called	isPending = true
pending	success	Mutation resolves	isPending = false, data = result
pending	error	Mutation rejects	isPending = false, isError = true

8.5 Manual guide ↔ Hook guide cross-reference

Manual testing scenario	Corresponding hook test	Connection
F-02: Create user with valid data	T-11: Create user success	Manual tests the UI flow; hook test verifies the mutation logic
BUG-UM-001: Duplicate on rapid save	E-13: Rapid state updates	Same root cause; hook test catches it at the logic level

BUG-UM-002: Success toast on 401	T-12: Create user API error	Hook test verifies error state is set, not success
BUG-UM-003: Sort resets on pagination	T-06: Cache prevents duplicate fetch	Both test caching behaviour from different angles
BUG-UM-004: Empty state not shown	E-02: Empty values	Hook returns [] so UI can show empty state
Edge: Network interruption	E-08: Error state + E-10: Retry	Hook test simulates network error at API level
Edge: Multiple tabs	E-11: Concurrent requests	Both test what happens with parallel operations
Edge: Browser refresh	E-14: Cleanup on unmount	Both test lifecycle management

KEY TAKEAWAY

The manual testing guide and this hook testing guide are two halves of the same strategy. Manual testing catches UI/UX issues that automated tests cannot. Hook tests catch logic issues that manual testing misses (race conditions, cache invalidation, state transitions). Use both together for full coverage.

Vitest Custom Hook Testing Guide for Freshers v1.0 — Running example: useUsers() hook (User Management feature). Companion to the Manual Testing Guide for Freshers.